LITE-SNN: Leveraging Inherent Dynamics to Train Energy-Efficient Spiking Neural Networks for Sequential Learning

Nitin Rathi and Kaushik Roy, *Fellow, IEEE*

***Abstract*—Spiking neural networks (SNNs) are gaining popularity for their promise of low-power machine intelligence on event-driven neuromorphic hardware. SNNs have achieved comparable performance as artificial neural networks (ANNs) on static tasks (image classification) with lower compute energy. In this work, we explore the inherent dynamics of SNNs for sequential tasks such as gesture recognition, sentiment analysis, and sequence-to-sequence learning on data from dynamic vision sensors (DVSs) and natural language processing (NLP). Sequential data are generally processed with complex recurrent neural networks (RNNs) [long short-term memory/gated recurrent unit (LSTM/GRU)] with explicit feedback connections and internal states to handle the long-term dependencies. The neuron models in SNNs—integrate-and-fire (IF) or leaky-integrate-and-fire (LIF)—have internal states (membrane potential) that can be efficiently leveraged for sequential tasks. The membrane potential in the IF/LIF neuron integrates the incoming current and outputs an event (or spike) when the potential crosses a threshold value. Since SNNs compute with highly sparse spike-based spatiotemporal data, the energy/inference is lower than LSTMs/GRUs. We also show that SNNs require fewer parameters than LSTM/GRU resulting in smaller models and faster inference. We observe the problem of vanishing gradients in vanilla SNNs for longer sequences and implement a convolutional SNN with attention layers to perform sequence-to-sequence learning tasks. The inherent recurrence in SNNs, in addition to the fully parallelized convolutional operations, provide additional mechanisms to model sequential dependencies that lead to better accuracy than convolutional neural networks (CNNs) with ReLU activations. We evaluate SNN on gesture recognition from the IBM DVS dataset, sentiment analysis from the IMDB movie reviews dataset, and German-to-English translation from the Multi30k dataset.**

***Index Terms*—Membrane potential, natural language processing (NLP), sequential learning, spiking neural networks (SNNs).**

Manuscript received 31 January 2023; revised 28 July 2023 and 5 January 2024; accepted 1 April 2024. Date of publication 3 May 2024; date of current version 10 December 2024. This work was supported in part by the National Science Foundation, and in part by the Center for the Codesign of Cognitive Systems (CoCoSys), one of seven centers in JUMP2.0, through Semiconductor Research Corporation (SRC) and DARPA. *(Corresponding author: Nitin Rathi.)*

The authors are with the School of Electrical and Computer Engineering, Purdue University, West Lafayette, IN 47907 USA (e-mail: rathi2@purdue.edu; kaushik@purdue.edu).

Digital Object Identifier 10.1109/TCDS.2024.3396431

I. Introduction

DEEP learning is achieving significant milestones in various fields including computer vision, natural language processing (NLP), drug discovery, autonomous driving, and many others. However, the success comes at a significant power and energy cost [1], and if unaddressed, it can hinder the deployment of ubiquitous intelligent edge devices, stagnate or slow down the progress of large model development, and affect various sustainability goals. Specifically, large language models (with trillions of parameters [2]) have recently received a lot of scrutiny for their high energy consumption [3]. Researchers are exploring various alternatives at the hardware (custom chips [4]), algorithms (model compression [5] and transfer learning [6]), and system-level [7] to circumvent the high energy and compute requirements of deep networks. Spiking neural networks (SNNs) running on low-power event-driven neuromorphic platforms [8] could be one such solution to improve the energy-efficiency of deep neural networks. SNNs are becoming popular for their promise of low-power machine intelligence through the asynchronous event-driven computations with binary signals (spikes). SNNs have achieved state-of-the-art accuracy on challenging image classification tasks with better energy efficiency compared to traditional (with ReLU like activations) artificial neural networks (ANNs) [9], [10], [11]. However, their application in other areas such as sequential learning tasks (speech recognition, language translation, sentiment analysis, etc.) is not well explored. In this work, we study the relation between SNNs and recurrent neural networks (RNNs) and focus on SNN's inherent recurrence dynamics in membrane potential and how it can be leveraged to store past information in sequential inputs. Moreover, we demonstrate that SNNs perform at a lower energy and memory budget compared to different RNN models such as long short-term memory (LSTM) [12] and gated recurrent unit (GRU) [13].

Recent improvements in SNN training mechanisms such as direct input coding [10], [11], [14], surrogate gradient-based backpropagation [15] have reduced the inference latency from thousands of time-steps to one time-step [10], [11], [16]. The low inference latency combined with sparse spike-based communication makes SNNs the perfect candidate to solve sequential learning tasks on energy-constrained devices. Although transformer-based models achieve excellent translation quality

on NLP tasks [17], they are too big to deploy on edge devices to achieve acceptable battery life. The other alternative is to use LSTMs, GRUs, and more recently, convolutional neural networks (CNNs) [18]. In this work, we compare the performance of SNNs with these alternatives to find an energy-efficient lightweight solution for NLP tasks on edge devices.

The rest of the article is organized as follows. First, Section II provides the background knowledge on SNNs discussing the neuron model, inherent recurrence, and the training mechanism. Next, Sections IV, V, and VI demonstrate the application of SNNs in gesture recognition, sentiment analysis, and sequence-to-sequence (seq2seq) learning tasks, respectively. Section VII compares the energy-efficiency of SNNs with RNN and ANN. Finally, we discuss the outlook and real-world applications of SNNs in Section VIII.

II. Background

One of the first works on training large scale SNNs for complex image classification tasks was shown in [19] on rate coding of inputs. Rate coding of inputs led to large latency for inference. Since then, SNNs have demonstrated comparable performance to traditional CNNs on image classification tasks with direct input coding as mentioned earlier while consuming significantly lower energy [10], [11], [20]. Despite the remarkable success of SNNs in image classification tasks and their energy-efficient nature of computing, they have not been extensively explored in the context of sequential learning tasks. Leveraging the inherent dynamics of SNNs to train energy-efficient networks presents a promising direction to address the energy consumption challenges in sequential learning tasks. The inherent advantage of SNNs lies in their ability to naturally process temporal information, making them better suited for sequential tasks. Unlike traditional networks that operate on continuous-valued activations, SNNs process information through discrete spikes, which inherently encode temporal dynamics. This event-driven behavior allows SNNs to capture and process sequential patterns in a more biologically plausible manner [21]. By exploiting the temporal nature of SNNs, we can unlock their potential for sequential learning tasks, such as NLP and time-series prediction, and pave the way for energy-efficient and high-performance solutions in these domains.

Existing works have established the viability of SNNs in specific contexts, such as speech recognition, showcasing comparable performance with LSTMs while delivering improved energy efficiency [22]. Furthermore, recent works by researchers have explored diverse applications, illustrating the versatility of SNNs. For instance, Lv et al. [23] present a two-step method for training SNNs in text classification, incorporating a simple yet effective approach to encode pre-trained word embeddings as spike trains. In [24], Luo et al. propose a novel method for utilizing SNNs in emotion classification from EEG signals, expanding the scope of SNN applications to neuroscientific domains. Additionally, Barchid et al. [25] showcase the efficacy of SNNs coupled with event cameras for facial expression recognition, demonstrating a remarkable 65x lower energy consumption compared to traditional ANNs. These seminal works underscore the potential of SNNs across diverse applications and inspire the present research to further amplify the impact of SNNs in real-world scenarios, where their energy efficiency can significantly reshape the landscape of intelligent computing. However, there is a need to extend the applicability of SNNs to a broader range of sequential tasks and datasets, addressing scalability challenges and ensuring generalization across diverse domains. This work seeks to expand the application of SNNs on other sequential datasets.

SNNs can be trained using both supervised and unsupervised learning techniques. Unsupervised learning involves algorithms that discover patterns in unlabeled data, without corresponding labels or targets. This approach is particularly useful for identifying clusters in raw and unknown data. Spike timing-dependent plasticity (STDP) is a biologically plausible unsupervised learning technique used for synaptic learning in SNNs [26]. STDP-based learning rules modify synaptic weights based on the correlation between spike times of pre- and postsynaptic neurons. There are different variants of STDP, with some rules performing both potentiation and depression based on the temporal differences between spikes. Initially, SNNs based on unsupervised STDP showed promising results for simple tasks such as digit recognition on the MNIST dataset. However, they faced limitations when applied to complex tasks such as image recognition on the ImageNet dataset. As a result, recent advancements in SNN research have shifted their focus toward the development of more powerful supervised learning algorithms to tackle these complex tasks effectively.

In this article, we propose a novel approach that harnesses the advantages of SNNs to train energy-efficient networks for sequential learning. We exploit the inherent recurrence in SNNs to process sequential data efficiently while maintaining competitive performance with state-of-the-art traditional neural networks. Our experimental results demonstrate the effectiveness of the proposed method on various sequential learning tasks, showcasing the potential of SNNs in achieving energy-efficient and high-performance artificial intelligence (AI) systems.

III. Method

In this section, we explain the neuron models employed in all the SNNs developed in this work, discuss the recurrence dynamics in SNN and its similarities to RNNs, and review the input coding and learning rules used to train the SNNs.

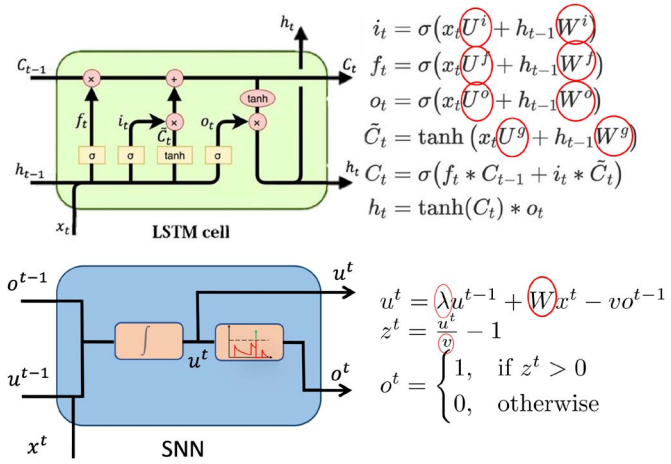
A. Neuron Model

We employ the IF/LIF neuron model described by

$$u_i^t = \lambda_i u_i^{t-1} + \sum_j w_{ij} o_j^t - v_i o_i^{t-1} \tag{1}$$

$$z_i^{t-1} = \frac{u_i^{t-1}}{v_i} - 1 \quad \text{and} \quad o_i^{t-1} = \begin{cases} 1, & \text{if } z_i^{t-1} > 0 \\ 0, & \text{otherwise} \end{cases} \tag{2}$$

where u is the membrane potential, λ is the leak factor with a value in $[0-1]$, w is the weight connecting preneuron j and postneuron i, o is the binary spike output, v is the firing threshold, and t represents the timestep. For IF neurons, leak (λ)



$$i_t = \sigma\left(x_t U^i + h_{t-1} W^i\right)$$
$$f_t = \sigma\left(x_t U^f + h_{t-1} W^f\right)$$
$$o_t = \sigma\left(x_t U^o + h_{t-1} W^o\right)$$
$$\tilde{C}_t = \tanh\left(x_t U^g + h_{t-1} W^g\right)$$
$$C_t = \sigma\left(f_t * C_{t-1} + i_t * \tilde{C}_t\right)$$
$$h_t = \tanh(C_t) * o_t$$

$$u^t = \lambda u^{t-1} + W x^t - v o^{t-1}$$
$$z^t = \frac{u^t}{v} - 1$$
$$o^t = \begin{cases} 1, & \text{if } z^t > 0 \\ 0, & \text{otherwise} \end{cases}$$

Fig. 1. SNN has 8× fewer weight parameters compared to LSTM. Although the leak factor (λ) and firing threshold (v) are also learnable parameters, they are shared by all neurons in the same layer, so the increase is trivial compared to the number of weight parameters.

is equal to 1. The second term in (1) integrates the inputs and the last term resets the membrane potential after firing. The reset mechanism reduces the membrane potential by the threshold value instead of resetting it to the reset potential. This reduces information loss and leads to better performance [27]. There are hardware implementations of LIF models that provide the functionality of reset by subtraction [28].

B. Inherent Recurrence Dynamics in SNNs

In this section, we study the similarities between RNNs and SNNs [15]. The term RNN refers to networks that have an explicit feedback connection in their internal state to store the history of previous inputs such as LSTMs, and GRUs as well as vanilla RNNs [29]. Fig. 1 compares the recurrent dynamical equations of LSTM and SNN. Each LSTM layer has eight weight matrices (similarly GRU has six, and vanilla RNN has two) compared to one weight matrix for SNN. Therefore, in terms of parameters, SNNs have the least number of trainable weights and require comparatively less storage. Both SNN and RNN receive time-varying input, have an internal state that serves as memory, and generate time-varying output. However, they differ in the type of recurrence. In RNN, the recurrent connection in the hidden state (h_t) is explicitly defined by a set of weight matrices; whereas, in SNN, the recurrence in membrane potential is implicitly defined via leak (λ). The leak is a single float value between 0 and 1 (shared by all the neurons in the same layer) that controls how much of the information encoded in the membrane potential from previous inputs/time-steps is carried forward to the next time-step (1). SNNs draw their strength from distributing computations over time. The spikes are distributed over T time-steps and one forward pass in RNN results in T forward passes in SNNs. However, each forward pass performs a simple addition operation on sparse binary activations and overall requires less energy (discussed in Section VII). We exploit this inherent membrane potential dynamics to design SNNs that can compete with RNNs and requires less storage and computational energy.

C. Input Coding and Training Mechanism

SNNs compute and communicate information through binary signals (spikes). Therefore, analog inputs such as image pixel and word embedding need to be converted to spike trains. There are various encoding methods such as rate coding [19], temporal coding [30], and direct coding [14]. In rate coding, the analog value is represented as the average firing rate of the neuron, whereas temporal coding encodes the analog value as the time difference between two successive spikes. Direct coding trains a neural network with LIF neurons that accept analog values as input and generates an output spike train. SNNs trained with direct coding have outperformed other coding methods in terms of latency and energy for image classification [10]. Thus, we employ the direct coding method in all SNNs developed in this work.

The training mechanisms for SNNs can be broadly classified into two categories: ANN-to-SNN conversion [19] and surrogate gradient-based backpropagation [31]. In ANN-to-SNN conversion, a shadow ANN is trained with gradient-descent and the weights are transferred to SNN followed by threshold balancing to determine the firing threshold of each layer. It takes advantage of all the training mechanisms available for ANNs, and the conversion to SNN is fast and efficient. However, the inference latency (the number of time-steps) for SNNs trained with this method is very high (>2000 time-steps) [19]. Alternately, SNNs can be trained with gradient-descent-based methods directly and can achieve lower latency (100 time-steps). Although the derivative of the LIF neuron is discontinuous and standard gradient-descent methods cannot be applied directly, many surrogate gradients [15] are proposed to approximate the true gradient and enable spike-based backpropagation training in SNNs. Rathi et al. [32] combined the ANN-to-SNN conversion and gradient-based training, that was further extended by Rathi and Roy [10] to include optimization of threshold voltage and leak along with weights of the network. SNNs trained with this method achieved very low latency (less than ten time-steps) and high accuracy, and therefore, we employ this training method in all the SNNs developed in this work. For all the experiments, we employ the cross-entropy loss and update the weights based on stochastic gradient descent. We use a batch size of 64, with Adam optimizer and learning rate of 0.001 that is decayed by 0.2 every $N/3$ steps, where N is the total number of epochs. More details are available in the source code at https://github.com/nitin-rathi/LITE-SNN.

In the following sections, we design and train SNNs that employ the input coding and training mechanisms discussed above and exploit the inherent recurrent dynamics to solve various sequential tasks.

IV. Gesture Recognition

Event-based sensors [37] capture the relative motion between the object and the camera. The output of the camera is binary, representing the presence of relative motion. Therefore, these

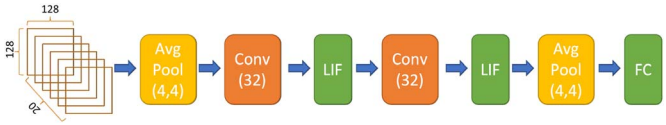


Fig. 2. SNN architecture for gesture recognition. Each gesture in the IBM DVS128 dataset is represented by a $128 \times 128 \times 20$ binary image. The sampled events are evenly distributed among the 20 channels.

TABLE I
PERFORMANCE COMPARISON BETWEEN THE PROPOSED METHOD AND OTHER METHODS ON IBM DVS128 DATASET

Model	Accuracy	Timesteps
SLAYER [33]	93.65%	300
DECOLLE [34]	95.54%	500
BPTT [35]	93.40%	1500
PointNet++ (ANN) [36]	94.10%	N/A
Our model	95.14%	5

sensors directly generate spikes and are suitable to be processed by spiking networks. Event cameras can be employed in optical flow estimation [38], gesture recognition [39], object tracking [40], and many other applications. In gesture recognition, the task is to identify the hand gestures from a series of event camera outputs. Each event is represented as $(time, x, y, polarity)$, where *time* is the time of the event, x, y is the location in the frame, and *polarity* is a binary value representing the change at the pixel location. A single gesture consists of thousands of such events. There are many ways [41] to represent a gesture so that it can be processed by a neural network. We select a certain set of events and evenly distribute the events into 20 blocks. All the events in one block are represented by a 128×128 frame where each location represents the presence of an event for that location in that block (Fig. 2). The 20 blocks are combined along the channel dimension, and the entire gesture is represented as $128 \times 128 \times 20$ binary image. We processed the gesture by an SNN with two conv layers, two pooling layers, and LIF neurons. We train the network on the IBM DVS128 dataset [41] and report the accuracy and inference latency in Table I. Our model achieves comparable accuracy as ANN and performs better in terms of latency than previous SNN models. In this method, the processing of a binary stream of data is performed by subsequently transforming it into an image format, incorporating temporal information representation along the channel dimension. The adoption of this approach proves particularly advantageous for batch inference scenarios. Nevertheless, for its successful implementation in online use cases, a prerequisite entails storing the binary stream in a memory buffer, followed by its conversion to an image format, which is subsequently presented as input to the network. This choice was informed by its ability to minimize the required number of time-steps, thus directly impacting computational load and enhancing overall energy efficiency in the system.

The membrane potential of the LIF neurons stores the information about previous time-steps and the leak parameter controls the flow of this information. Additionally, the threshold determines if the spike should propagate in the current time-step. By training the threshold and leak with gradient-descent,

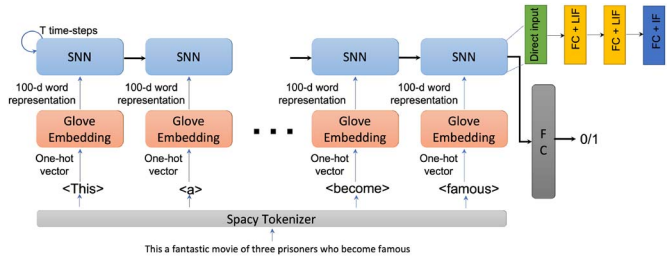


Fig. 3. Unrolled SNN architecture for sentiment analysis. Each input token is processed by the SNN in five time-steps. The membrane potential of the IF/LIF neurons holds the processed information about past tokens. The membrane potential of the final FC layer (one neuron) tracks the overall sentiment over time as each token is processed.

we learn the optimal amount of past information to store and propagate spikes at the right time-step that leads to better accuracy with lower latency.

V. SENTIMENT ANALYSIS

In this section, we explore the application of SNNs for NLP tasks that have traditionally been solved using RNNs. Sentiment analysis is a binary classification problem where given an input text, the task is to classify the text as having positive or negative sentiment. The IMDB dataset has 50 000 labeled movie reviews with 25 000 reviews used for training and 25 000 for testing. Fig. 3 shows the text preprocessing and unrolled SNN architecture employed for sentiment analysis. The spacy tokenizer[1] splits the input sentence into individual tokens that are one-hot encoded. Next, a pretrained glove embedding [44] is employed to generate a dense vector representation of the input token. The dense vector is processed by the SNN. SNN has two fully connected (FC) layers with LIF neurons and one FC layer with IF neurons. Fig. 4 shows the change in membrane potential of the final FC layer IF neuron. We employ IF/LIF neuron models because they are simple, require less number of computations to update the membrane potential, and the surrogate-gradient-based learning algorithms are well defined for these models. The state of the SNN is preserved after processing each token, in other words, the membrane potentials of LIF and IF neurons are not reset. The history of all the tokens is stored in the membrane potential of LIF and IF neurons. Fig. 5 compares the performance of vanilla RNN, LSTM, and SNN. We performed experiments with different number of parameters starting from smallest architecture where all three models underfit to larger sizes where the models overfit. For 60 k parameters, SNN achieves its highest accuracy of 88.51%, whereas LSTM reaches 87.60%. LSTM achieves the highest accuracy of 89.32% at 250k parameters. Table II shows the min, max, and average accuracy for model sizes when each model achieves its highest accuracy. The results suggest that SNNs can provide a better alternative than LSTMs to solve NLP tasks on energy-constrained edge devices. Table III shows the accuracy reported in other works that closely match the results of our models.

The sentiment is encoded in the membrane potential of the final layer neuron (Fig. 4), which stores the information

[1]https://spacy.io/

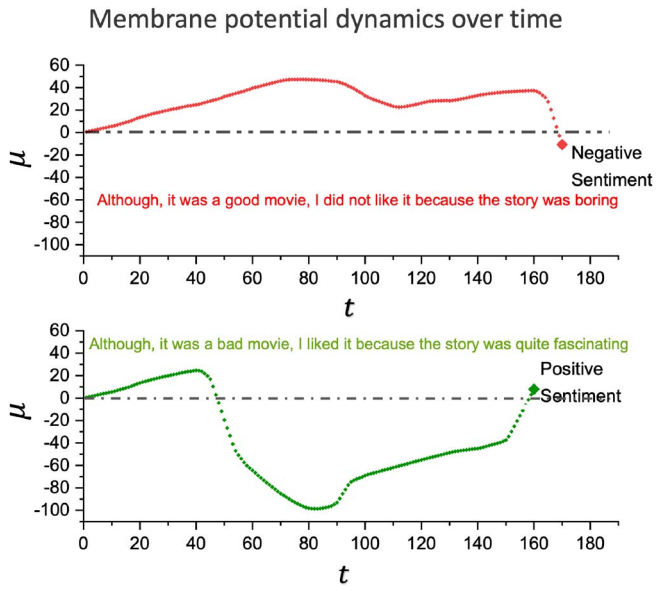


Fig. 4. Change in membrane potential over time of output layer with one (binary classification) IF neuron for two different inputs. The membrane potential at any time represents the sentiment of all previous words processed till that time. In the top example, there are positive words ("good movie") in the beginning and therefore the membrane potential is high, however, the membrane potential goes down in the end to reflect the overall negative sentiment of the input and vice-versa for the bottom example.

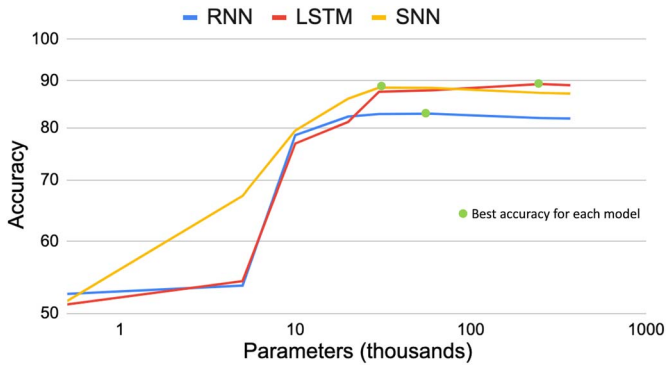


Fig. 5. Accuracy comparison between RNN, LSTM, and SNN for different number of parameters. Each data point is obtained as the average of five runs with different random initialization of weights. The best average accuracy for each model is shown by the green circle and also mentioned in Table II.

TABLE II
PERFORMANCE COMPARISON ON IMDB DATASET BETWEEN VANILLA RNN, LSTM, AND SNN

Model	Parameters (thousands)	Accuracy (min)	Accuracy (max)	Accuracy (avg)
Vanilla RNN	60	82.77%	82.89%	82.84%
LSTM	250	89.28%	89.36%	89.32%
SNN	30	88.38%	88.53%	88.51%

Note: Each experiment is performed with five different random initialization; min, max, and average values are shown in the table.

about all the tokens processed till the current time-step. The membrane potential rises for tokens that have positive sentiment ("good" and "fascinating") and falls for negative sentiment ("bad" and "boring"). The membrane potential tracks the real-time sentiment as the tokens are processed, and therefore, for the negative example, it rises initially because of positive tokens ("good") but eventually falls down at the end due to the overall negative sentiment.

TABLE III
PERFORMANCE COMPARISON ON IMDB DATASET WITH OTHER SOTA MODELS

Model	Accuracy
LSTM+CNN [42]	89.5%
MLP [43]	87.0%
Our model (SNN)	88.54%

VI. SEQUENCE-TO-SEQUENCE LEARNING

In sequence-to-sequence learning tasks, a given sequence of arbitrary length is transformed into another sequence of arbitrary length [45]. For example, summarizing a long paragraph into few sentences, translating text from one language to another, etc. In this section, we will discuss the language translation task (German to English) with vanilla SNN, SNN with attention layers, and convolutional SNN with attention. Each architecture tries to overcome the shortcomings of the previous architecture and we get the best performance from the convolutional SNN.

A. Vanilla SNN

The most common sequence-to-sequence models are encoder–decoder models [45]. The encoder, usually an RNN, encodes the input sequence into a dense vector known as the context vector. The context vector is a representation of the entire input sequence. The decoder, also an RNN, generates the output sequence from the context vector, one word at a time. We replace the RNNs with SNNs in both the encoder and decoder and train the model for German-to-English translation from Multi30k dataset [46]. Fig. 6 shows the network architecture with SNN in both encoder and decoder. We also experiment with a combination of GRU and SNN as encoder and decoder (Table IV). We employ BLEU score [47] as the evaluation metric and consider N-grams (with $N = 4$) using uniform weights, commonly known as BLEU-4. The model with both encoder and decoder as GRUs performs the best, whereas the model with SNNs fails to train. The issue of vanishing/exploding gradient is well known for recurrent networks [48] including SNNs that have inherent recurrence [22]. The issue is exacerbated in SNNs with two additional factors: 1) time-steps; and 2) surrogate gradient. The number of time-steps further increases the length of the sequence as each word is processed for five time-steps. The surrogate gradient is an approximation and long sequences result in many such approximations being multiplied together leading to unstable weight updates. We also observe that encoders have a more significant role than decoders (Table IV) because replacing GRU with SNN in the encoder is worse than doing the same for the decoder (BLEU score 16 versus 20).

B. SNN With Attention

The vanilla encoder–decoder model suffers from information compression as the entire input sequence is represented by a

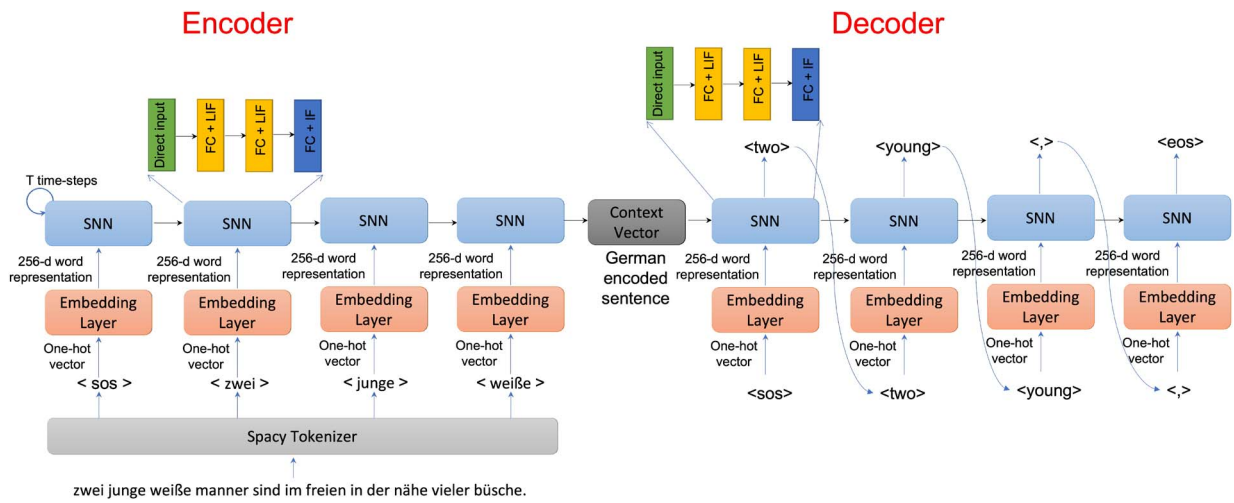


Fig. 6. Encoder–decoder architecture with SNNs for language translation. The German sentence is encoded in the membrane potential of the IF/LIF neurons and the decoder reads the membrane potential at each time-step to generate the English translation one token at a time.

TABLE IV
PERFORMANCE ON GERMAN-TO-ENGLISH TRANSLATION WITH VANILLA SNNS

Encoder	Decoder	BLEU Score
GRU	GRU	22
SNN	GRU	16
GRU	SNN	20
SNN	SNN	Unable to train

single vector. To address this issue, researchers have proposed an attention mechanism to allow the decoder to look at the encoder's intermediate representations at each decoding step [49]. An attention vector (a) with same length as the input sequence is used to compute a weighted context vector (z) from all the hidden states (h) of the encoder ($z = \sum a_i h_i$). Each element (a_i) in the attention vector is between 0 and 1 and the entire vector sums to 1. The attention vector is recomputed at every decoding step. This allows the decoder to focus on different input words to generate each output word. The attention vector (a) is computed as a function of previous decoder state (s_{t-1}) and all encoder hidden states (h) [$a_t = f(s_{t-1}, h)$]. The elements in the attention vector tell us how much we should attend to each word in the source sentence. For more information on the attention mechanism we refer to [49].

We introduce the attention mechanism to our vanilla SNN models (Fig. 7) and treat the accumulated membrane potentials of IF neurons as intermediate hidden states. Similarly, the membrane potential of the decoder SNN at the previous time-step is used to compute the attention for the current time-step. We compare the performance of SNN with GRU for different encoder–decoder combinations (Table V). Compared to vanilla SNNs, we observe that the attention mechanism alleviates the vanishing gradient problem and we can train the model when both the encoder and decoder are SNNs. However, the performance of SNN is worse compared to GRUs. The primary reasons are vanishing gradient and vanishing spike problem.

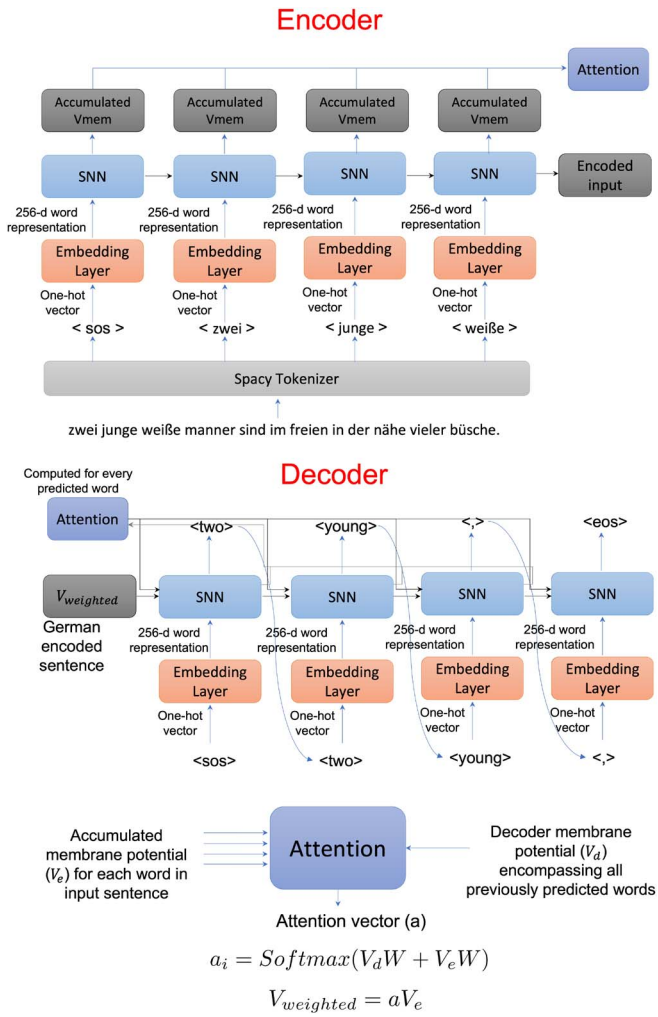


Fig. 7. Network architecture for language translation with attention and SNNs. The attention vector is computed from the membrane potential of the encoder and decoder. The attention vector is then used to compute a weighted membrane potential from all hidden states of the encoder.

TABLE V
PERFORMANCE ON GERMAN-TO-ENGLISH TRANSLATION WITH SNN AND ATTENTION

Encoder	Decoder	BLEU Score
GRU	GRU	33
SNN	GRU	19
GRU	SNN	24
SNN	SNN	15

The spiking activity in SNNs decreases drastically for deeper layers because the accumulation in IF/LIF neurons results in lower output spikes than input spikes for each layer (vanishing spike problem). For longer sequences, the spiking activity decreases for latter words and accompanied by the vanishing gradient problem makes the issue worse. Therefore, applying SNNs in models that process words sequentially may not be efficient. In the next section, we discuss how we can employ convolutional SNNs to alleviate the problem.

C. Convolutional SNN With Attentions

RNNs combined with attention mechanisms have achieved great success in solving sequence-to-sequence learning tasks [49]. However, these models suffer from gradient propagation and cannot be fully parallelized over the input sequence. On the other hand, CNNs have achieved significant success in computer vision tasks and the computations are fully parallelized. In recent years, several CNN-based models have been proposed for sequence-to-sequence learning that addresses the shortcomings of RNNs [18]. Also, several convolutional SNN models perform extremely well on image classification tasks [10], [19]. Combining the success of training SNNs for image classification with CNN-based sequence-to-sequence models, we propose a convolutional SNN model for the language translation task. Fig. 8 shows the network architecture with 1-D conv layers and LIF neurons in both encoder and decoder. The final layer in both encoder and decoder consists of integrate neurons (IF neurons with very high threshold to avoid firing) to accumulate spikes from all time-steps. Since all the input words are processed in parallel, positional embedding is added to encode the order of the words within a sequence. We still employ the attention mechanism (not shown in Fig. 8) on all encoder hidden states to compute the context vector. The decoder also processes all the target words in parallel and therefore, to ensure that the filters translating token i only look at tokens that appear before i, we add padding only at the beginning of the sentence. As shown in Fig. 8, to predict the word "two" the decoder looks at two padding tokens and the "sos" token. If the padding was distributed equally in the beginning and end, the conv kernel would look at the word it is trying to predict and will simply learn to copy it instead of learning to translate it. We compare the performance of SNN with conv layers and LIF neurons with an ANN having a similar number of conv layers and ReLU activation. Tables VI and VII show the performance of ANN and SNN with encoder and decoder each having five and ten convolutional layers, respectively. Unlike the $6-8\times$ benefit in the number of parameters as compared to RNNs, the number of parameters in ANN and SNN are similar for the same number of conv layers. SNN has slightly more parameters due to membrane potential and leak. Each neuron has its own membrane potential, whereas leak is shared by all neurons in the same layer. The SNN with five conv layers performs similar to the ANN with ten conv layers. Note, we achieve $2\times$ benefit in the number of parameters (due to lesser number of layers) for the same BLEU score. SNN's better performance comes from its inherent recurrence in membrane potential. Although convolutional SNNs process the sequence in parallel, the inherent recurrence in SNNs stores the history of nearby words in its membrane potential and provides the benefit of both parallel processing and recurrence. It also solves the spike/gradient vanishing problem by reducing the gradient propagation path to the sum of the number of layers and time-steps. Thus, we believe, SNNs are more suitable for solving sequential tasks with convolutional layers and achieve similar performance as ANNs with $2\times$ less number of parameters.

VII. ENERGY EFFICIENCY

SNNs exhibit superior energy efficiency when compared to traditional ANNs, primarily owing to the nature of their operations. In SNNs, each computation involves a single addition (AC), as opposed to the multiplication and addition operations (MAC) inherent in ANNs. This distinction arises from the binary nature of spikes in SNNs, where computations are reduced to simple additions. Furthermore, the implementation of SNNs on neuromorphic hardware, as exemplified by architectures such as Loihi [8] and other neuromorphic chips [50], capitalizes on event-driven processing. In the absence of spikes, no computations are performed, leading to a cessation of active energy consumption. This contrasts sharply with ANNs, where continuous computation is prevalent, resulting in a consistent energy expenditure. To precisely ascertain the energy demands of SNNs, it is imperative to deploy these networks on specialized neuromorphic hardware capable of capturing both compute and memory access energy. However, as the primary emphasis of this study lies in investigating algorithms and model architectures, a comprehensive hardware implementation of SNNs was not undertaken. For a deeper understanding of hardware implementations of the algorithms under consideration, we direct the readers to relevant articles that discuss these aspects [28], [51].

In 45 nm CMOS technology, the energy cost of a 32-bit MAC operation is reported to be $5.1\times$ higher than that of an AC operation [52]. Although these values may vary across different technologies, the fundamental notion that addition operations are significantly more energy efficient than multiplication operations remains consistent. The implications of these energy dynamics are profound, particularly in the realm of AI hardware. SNNs, with their innate energy efficiency and event-driven processing, demonstrate promising potential for realizing energy-efficient AI solutions. A noteworthy illustration is provided by Jin et al. [53], wherein an SNN implementation on a neuromorphic chip showcased a remarkable $280\times$ improvement in energy efficiency compared to ANN implementations on GPUs.

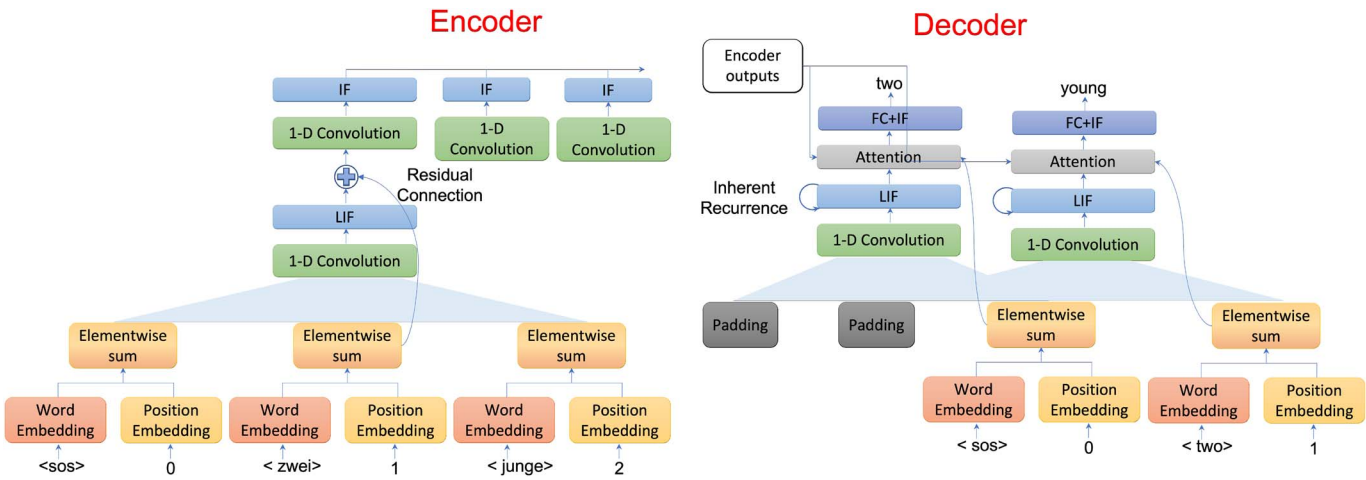


Fig. 8. Network architecture for language translation with convolutional SNNs. Position embedding is added along with word embedding to encode the order of tokens as all input words are processed parallelly. The IF neuron accumulates the information in the membrane potential from each time-step. All the tokens are processed in five time-steps.

TABLE VI
GERMAN-TO-ENGLISH TRANSLATION WITH CONVOLUTIONAL SNN AND ATTENTION

Encoder	Decoder	BLEU Score
ANN	ANN	32
SNN	ANN	34
ANN	SNN	34
SNN	SNN	36

Note: Encoder and decoder consist of five conv layers.

TABLE VII
GERMAN-TO-ENGLISH TRANSLATION WITH CONVOLUTIONAL SNN AND ATTENTION

Encoder	Decoder	BLEU Score
ANN	ANN	36
SNN	ANN	34
ANN	SNN	35
SNN	SNN	36

Note: Encoder and decoder consist of ten conv layers.

In conclusion, the distinctive characteristics of SNNs, coupled with their implementation on neuromorphic hardware, underscore their role as a catalyst for advancing energy-efficient AI.

VIII. DISCUSSION

SNNs have achieved decent performance on image classification tasks, and recent developments in training methodologies have improved their inference latency and energy. However, the true potential of SNNs can be realized for sequential tasks. SNNs are dynamical systems and are naturally better suited to handle sequential inputs than static images. We showed, through experiments, on various NLP tasks, how SNNs provide advantages over LSTMs/GRUs.

Despite the strides made in leveraging SNNs for sequential tasks with improved energy and memory efficiency, avenues for further exploration exist to enhance their capabilities. While the training methodologies for SNNs have progressed, optimizing their performance on complex tasks, particularly those involving extensive temporal dependencies, remains an area for continued investigation. Additionally, understanding the internal dynamics of SNNs in intricate sequential tasks requires attention, fostering interpretability and trust in their applications.

The remarkable energy efficiency exhibited by SNNs positions them as promising candidates for a myriad of real-world applications where resource constraints and sustainability are paramount considerations. In edge computing environments, SNNs could be employed for tasks such as sensor data processing and simple pattern recognition with limited memory, capitalizing on their low-energy consumption during periods of inactivity. Additionally, in Internet of Things (IoT) deployments, SNNs could play a pivotal role in enabling intelligent decision-making at the device level, minimizing the need for constant data transmission to centralized servers and thereby reducing overall energy consumption. Moreover, SNNs could find applications in energy-sensitive domains such as wearable devices, where the efficient processing of sequential data, such as physiological signals, can be pivotal for real-time health monitoring without unduly draining device batteries. By harnessing the energy-efficient nature of SNNs in these real-world scenarios, there exists a tangible potential to revolutionize the landscape of edge computing, IoT, and wearable technology, paving the way for sustainable and adaptive AI systems. However, there still remain challenges for faster training of SNNs.

DATA AVAILABILITY STATEMENT

The source code is available at https://github.com/nitin-rathi/LITE-SNN.

REFERENCES

[1] N. C. Thompson, K. Greenewald, K. Lee, and G. F. Manso, "Deep learning's diminishing returns: The cost of improvement is becoming unsustainable," *IEEE Spectr.*, vol. 58, no. 10, pp. 50–55, Oct. 2021.

[2] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," 2021, *arXiv:2101.03961*.
[3] E. Strubell, A. Ganesh, and A. McCallum, "Energy and policy considerations for deep learning in NLP," 2019, *arXiv:1906.02243*.
[4] A. Shawahna, S. M. Sait, and A. El-Maleh, "FPGA-based accelerators of deep learning networks for learning and classification: A review," *IEEE Access*, vol. 7, pp. 7823–7859, 2018, doi: 10.1109/ACCESS.2018.2890150.
[5] L. Deng, G. Li, S. Han, L. Shi, and Y. Xie, "Model compression and hardware acceleration for neural networks: A comprehensive survey," *Proc. IEEE*, vol. 108, no. 4, pp. 485–532, 2020.
[6] L. Torrey and J. Shavlik, "Transfer learning," in *Handbook of Research on Machine Learning Applications and Trends: Algorithms, Methods, and Techniques,* Hershey, PA, USA: IGI Global, 2010, pp. 242–264.
[7] B. Acun, M. Murphy, X. Wang, J. Nie, C.-J. Wu, and K. Hazelwood, "Understanding training efficiency of deep learning recommendation models at scale," in *Proc. IEEE Int. Symp. High-Perform. Comput. Archit. (HPCA)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 802–814.
[8] M. Davies et al., "Loihi: A neuromorphic manycore processor with on-chip learning," *IEEE Micro*, vol. 38, no. 1, pp. 82–99, Jan./Feb. 2018.
[9] J. Wu, Y. Chua, M. Zhang, G. Li, H. Li, and K. C. Tan, "A tandem learning rule for efficient and rapid inference on deep spiking neural networks," *IEEE Trans. Neural Netw. Learn. Syst.,* vol. 34, no. 1, pp. 446–460, Jan. 2023.
[10] N. Rathi and K. Roy, "DIET-SNN: A low-latency spiking neural network with direct input encoding and leakage and threshold optimization," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 6, pp. 3174–3182, Jun. 2023.
[11] S. S. Chowdhury, N. Rathi, and K. Roy, "Towards ultra low latency spiking neural networks for vision and sequential tasks using temporal pruning," in *Proc. Eur. Conf. Comput. Vis.*, Cham, Switzerland: Springer-Verlag, 2022, pp. 709–726.
[12] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Comput.*, vol. 9, no. 8, pp. 1735–1780, 1997.
[13] J. Chung, C. Gulcehre, K. Cho, and Y. Bengio, "Empirical evaluation of gated recurrent neural networks on sequence modeling," 2014, *arXiv:1412.3555*.
[14] B. Rueckauer, I.-A. Lungu, Y. Hu, M. Pfeiffer, and S.-C. Liu, "Conversion of continuous-valued deep networks to efficient event-driven networks for image classification," *Frontiers Neurosci.*, vol. 11, 2017, Art. no. 682.
[15] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks," 2019, *arXiv:1901.09948*.
[16] W. Zhang and P. Li, "Temporal spike sequence learning via backpropagation for deep spiking neural networks," 2020, *arXiv:2002.10085*.
[17] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, 2017, pp. 5998–6008.
[18] J. Gehring, M. Auli, D. Grangier, D. Yarats, and Y. N. Dauphin, "Convolutional sequence to sequence learning," in *Proc. Int. Conf. Mach. Learn.,* PMLR, 2017, pp. 1243–1252.
[19] A. Sengupta, Y. Ye, R. Wang, C. Liu, and K. Roy, "Going deeper in spiking neural networks: VGG and residual architectures," *Frontiers Neurosci.*, vol. 13, 2019.
[20] Y. Wu, L. Deng, G. Li, J. Zhu, Y. Xie, and L. Shi, "Direct training for spiking neural networks: Faster, larger, better," in *Proc. AAAI Conf. Artif. Intell.*, vol. 33, 2019, pp. 1311–1318.
[21] W. Maass, "Networks of spiking neurons: The third generation of neural network models," *Neural Netw.*, vol. 10, no. 9, pp. 1659–1671, 1997.
[22] W. Ponghiran and K. Roy, "Spiking neural networks with improved inherent recurrence dynamics for sequential learning," 2021, *arXiv:2109.01905*.
[23] C. Lv, J. Xu, and X. Zheng, "Spiking convolutional neural networks for text classification," in *Proc. 11th Int. Conf. Learn. Represent.*, 2022.
[24] Y. Luo et al., "EEG-based emotion classification using spiking neural networks," *IEEE Access*, vol. 8, pp. 46007–46016, 2020, doi: 10.1109/ACCESS.2020.2978163.
[25] S. Barchid, B. Allaert, A. Aissaoui, J. Mennesson, and C. C. Djeraba, "Spiking-FER: Spiking neural network for facial expression recognition with event cameras," in *Proc. 20th Int. Conf. Content-Based Multimedia Indexing*, 2023, pp. 1–7.
[26] C. Lee, G. Srinivasan, P. Panda, and K. Roy, "Deep spiking convolutional neural network trained with unsupervised spike-timing-dependent plasticity," *IEEE Trans. Cogn. Develop. Syst.*, vol. 11, no. 3, pp. 384–394, Sep. 2019.
[27] B. Han, G. Srinivasan, and K. Roy, "RMP-SNNS: Residual membrane potential neuron for enabling deeper high-accuracy and low-latency spiking neural networks," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2020, pp. 13558–13567.
[28] A. Agrawal, M. Ali, M. Koo, N. Rathi, A. Jaiswal, and K. Roy, "IMPULSE: A 65-nm digital compute-in-memory macro with fused weights and membrane potential for spike-based sequential learning tasks," *IEEE Solid-State Circuits Lett.*, vol. 4, pp. 137–140, 2021.
[29] T. Young, D. Hazarika, S. Poria, and E. Cambria, "Recent trends in deep learning based natural language processing," *IEEE Comput. Intell. Mag.*, vol. 13, no. 3, pp. 55–75, Aug. 2018.
[30] M. Zhang, Z. Gu, N. Zheng, D. Ma, and G. Pan, "Efficient spiking neural networks with logarithmic temporal coding," *IEEE Access*, vol. 8, pp. 98156–98167, 2020, doi: 10.1109/ACCESS.2020.2994360.
[31] G. Bellec, D. Salaj, A. Subramoney, R. Legenstein, and W. Maass, "Long short-term memory and learning-to-learn in networks of spiking neurons," in *Proc. Adv. Neural Inf. Process. Syst.*, 2018, pp. 787–797.
[32] N. Rathi, G. Srinivasan, P. Panda, and K. Roy, "Enabling deep spiking neural networks with hybrid conversion and spike timing dependent backpropagation," in *Proc. Int. Conf. Learn. Represent.*, 2020. [Online]. Available: https://openreview.net/forum?id=B1xSperKvH
[33] S. B. Shrestha and G. Orchard, "SLAYER: Spike layer error reassignment in time," in *Proc. Adv. Neural Inf. Process. Syst.*, Red Hook, NY, USA: Curran Associates Inc., 2018, pp. 1412–1421.
[34] J. Kaiser, H. Mostafa, and E. Neftci, "Synaptic plasticity dynamics for deep continuous local learning (DECOLLE)," *Frontiers Neurosci.*, vol. 14, 2020, Art. no. 424.
[35] W. He et al., "Comparing SNNS and MNS on neuromorphic vision datasets: Similarities and differences," *Neural Netw.*, vol. 132, pp. 108–120, 2020.
[36] Q. Wang, Y. Zhang, J. Yuan, and Y. Lu, "Space-time event clouds for gesture recognition: From RGB cameras to event cameras," in *Proc. IEEE Winter Conf. Appl. Comput. Vision (WACV),* Piscataway, NJ, USA: IEEE Press, 2019, pp. 1826–1835.
[37] L. Patrick, C. Posch, and T. Delbruck, "A 128x 128 120 db 15μs latency asynchronous temporal contrast vision sensor," *IEEE J. Solid-State Circuits*, vol. 43, no. 2, pp. 566–576, Feb. 2008.
[38] C. Lee, A. K. Kosta, and K. Roy, "Fusion-FlowNet: Energy-efficient optical flow estimation using sensor fusion and deep fused spiking-analog network architectures," 2021, *arXiv:2103.10592*.
[39] R. Massa, A. Marchisio, M. Martina, and M. Shafique, "An efficient spiking neural network for recognizing gestures with a DVS camera on the Loihi neuromorphic processor," 2020, *arXiv:2006.09985*.
[40] H. Liu, D. P. Moeys, G. Das, D. Neil, S.-C. Liu, and T. Delbrück, "Combined frame-and event-based detection and tracking," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS),* Piscataway, NJ, USA: IEEE Press, 2016, pp. 2511–2514.
[41] A. Amir, et al., "A low power, fully event-based gesture recognition system," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, 2017, pp. 7243–7252.
[42] A. Yenter and A. Verma, "Deep CNN-LSTM with combined kernels from multiple branches for IMDb review sentiment analysis," in *Proc. IEEE 8th Annu. Ubiquitous Comput., Electron. Mobile Commun. Conf. (UEMCON),* Piscataway, NJ, USA: IEEE Press, 2017, pp. 540–546.
[43] Z. Shaukat, A. A. Zulfiqar, C. Xiao, M. Azeem, and T. Mahmood, "Sentiment analysis on IMDb using lexicon and neural networks," *SN Appl. Sci.*, vol. 2, pp. 1–10, 2020.
[44] J. Pennington, R. Socher, and C. D. Manning, "GloVe: Global vectors for word representation," in *Proc. Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, 2014, pp. 1532–1543.
[45] I. Sutskever, O. Vinyals, and Q. V. Le, "Sequence to sequence learning with neural networks," in *Proc. Adv. Neural Inf. Process. Syst.*, Cambridge, MA, USA: MIT Press, 2014, pp. 3104–3112.
[46] D. Elliott, S. Frank, K. Sima'an, and L. Specia, "Multi30k: Multilingual English-German image descriptions," 2016, *arXiv:1605.00459*.
[47] K. Papineni, S. Roukos, T. Ward, and W.-J. Zhu, "Bleu: A method for automatic evaluation of machine translation," in *Proc. 40th Annu. Meeting Assoc. Comput. Linguistics*, 2002, pp. 311–318.
[48] S. Hochreiter, "The vanishing gradient problem during learning recurrent neural nets and problem solutions," *Int. J. Uncertainty, Fuzziness Knowl. Based Syst.*, vol. 6, no. 2, pp. 107–116, 1998.

[49] D. Bahdanau, K. Cho, and Y. Bengio, "Neural machine translation by jointly learning to align and translate," 2014, *arXiv:1409.0473*.
[50] E. P. Frady et al., "Neuromorphic nearest-neighbor search using intel's Pohoiki springs," 2020, *arXiv:2004.12691*.
[51] M. Zhang et al., "Rectified linear postsynaptic potential function for backpropagation in deep spiking neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 33, no. 5, pp. 1947–1958, May 2021.
[52] M. Horowitz, "1.1 computing's energy problem (and what we can do about it)," in *Proc. IEEE Int. Solid State Circuits Conf. Dig. Tech. Papers (ISSCC),* Piscataway, NJ, USA: IEEE Press, 2014, pp. 10–14.
[53] X. Jin, M. Zhang, R. Yan, G. Pan, and D. Ma, "R-SNN: Region-based spiking neural network for object detection," *IEEE Trans. Cogn. Develop. Syst.*, 2023, doi: 10.1109/TCDS.2023.3311634.

Nitin Rathi received the B.Tech. degree in electronics and communications engineering from West Bengal University of Technology, Kolkata, India, in 2013. He is currently working toward the Ph.D. degree in electrical and computer engineering with Purdue University, West Lafayette, IN, USA.

He worked as a Machine Learning Intern with GlobalFoundries, Santa Clara, CA, USA, in 2020, where he developed machine learning algorithms to identify defects from chip design images. His research interests include machine learning algorithms, neuromorphic computing, and energy-efficient deep learning.

Kaushik Roy (Fellow, IEEE) received the B.Tech. degree in electronics and electrical communications engineering from the Indian Institute of Technology, Kharagpur, India, in 1983, and the Ph.D. degree in electrical and computer engineering from the University of Illinois, Urbana-Champaign, IL, USA, in 1990.

He was with the Semiconductor Process and Design Center of Texas Instruments, Dallas, TX, USA, where he worked on FPGA architecture development and low-power circuit design. He joined the Electrical and Computer Engineering Faculty, Purdue University, West Lafayette, IN, USA, in 1993, where he is currently the Edward G. Tiedemann Jr. Distinguished Professor. He is also the Director of the Center for Brain-Inspired Computing (C-BRIC) funded by SRC/DARPA. His research interests include neuromorphic and emerging computing models, neuromimetic devices, spintronics, device-circuit-algorithm codesign for nanoscale silicon and nonsilicon technologies, and low-power electronics. He has published more than 700 papers in refereed journals and conferences, holds more than 25 patents, and supervised 90 Ph.D. dissertations. He is a co-author of two books titled *Low Power CMOS VLSI Design* (John Wiley and McGraw Hill, 2000).